

مستندات فنی پروژه فرانت‌اند

تکنولوژی‌های استفاده‌شده

فریم‌ورک و زبان

- **Next.js 16.1.1:** App Router فریم‌ورک اصلی پروژه بر پایه
- ****React 19.2.3**
- **TypeScript:** افزایش ایمنی نوع‌ها و کاهش خطاهای زمان اجرا

استایل و UI

- **Tailwind CSS v4:** برای استایل‌دهی Utility-first سیستم
- **shadcn/ui + Radix UI:** Accessible کامپوننت‌های قابل استفاده مجدد و
- **clsx / tailwind-merge:** مدیریت کلاس‌های داینامیک
- **lucide-react:** آیکون‌ها

مدیریت داده و State

- داده‌ها sync و refetch، سرور، کش state مدیریت: **@tanstack/react-query**
- **Zustand:** state کلاینت (UI state، auth، steps)

فرم و اعتبارسنجی

- **react-hook-form:** بالا performance مدیریت فرم‌ها با
- **zod:** TypeScript اعتبارسنجی اسکیمای هماهنگ با
- **@hookform/resolvers:** به فرم‌ها Zod اتصال
- Schema تعریف جداگانه برای هر فرم

ارتباط با API

- **Axios:** های سمت کلاینت API ارتباط با
- **Fetch:** های سمت سرور API ارتباط با

مدیریت پکیج (Package Manager)

-- **pnpm:** اصلی پروژه Package Manager به‌عنوان

دلایل استفاده از **pnpm:**

- سرعت بالاتر نسبت به yarn و npm
- مصرف فضای دیسک کمتر به دلیل ساختار content-addressable
- جلوگیری از نصب وابستگی‌های ناخواسته (strict dependency resolution)
- مناسب برای پروژه‌های مقیاس‌پذیر و تیمی

* استراتژی تست پروژه

روش به نام Co-located Tests** شناخته می‌شود

- ارتباط مستقیم تست با کد

- امن‌تر Refactor

- پیدا کردن تست بسیار سریع
- جلوگیری از تست‌های فراموش‌شده
- هماهنگی با Feature-based Architecture

در صورت پیاده‌سازی تست در آینده، ابزار **Vitest** به‌عنوان انتخاب مناسب برای تست‌های واحد و یکپارچه پیشنهاد می‌شود.

و هماهنگی مناسب با معماری پروژه، گزینه‌ای مناسب، **TypeScript** به دلیل سرعت بالا، سازگاری کامل با **Vitest** و **utility** برای تست منطق کسب‌وکار، هوک‌ها، فرم‌ها و

برای تست‌های End-to-End، ابزار **Playwright** پیشنهاد می‌شود.

معماری پروژه

Feature-based Architecture

پروژه بر اساس **feature**ها سازمان‌دهی شده است:

```
src/
├─ app/
│   ├── (main)/
│   ├── layout.tsx
│   └── page.tsx
├─ features/
│   └── auth/
```

```
|
|
|   ├── components/
|   |   ├── LoginForm.tsx
|   |   └── LoginForm.test.tsx
|   ├── hooks/
|   |   ├── useLogin.ts
|   |   └── useLogin.test.ts
|   ├── services/
|   |   ├── auth.api.ts
|   |   └── auth.api.test.ts
|   ├── schemas/
|   |   ├── login.schema.ts
|   |   └── login.schema.test.ts
|   ├── store/
|   |   ├── auth.store.ts
|   |   └── auth.store.test.ts
|   └── types/
|
└── product/
    ├── components/
    ├── hooks/
    └── ...

└── shared/
    ├── components/
    |   ├── Button.tsx
    |   └── Button.test.tsx
    ├── hooks/
    |   ├── useDebounce.ts
    |   └── useDebounce.test.ts
    └── utils/
        ├── formatDate.ts
        └── formatDate.test.ts

└── tests/
    ├── setup.ts
    ├── mocks/
    |   ├── server.ts
    |   ├── handlers.ts
    |   └── auth.handlers.ts
    └── utils/

└── docs/
```

- افزایش خوانایی
- مقیاس‌پذیری بهتر
- تست‌پذیری بالاتر

می‌شود.

استفاده از shadcn/ui

اصول

- اختصاصی برای هر کامپوننت Wrapper

```
// shared/components/Button.tsx
import { Button as ShadButton } from '@components/ui/button'

export function Button(props) {
  return <ShadButton {...props} />
}
```

مزایا:

- قابلیت تغییر UI در آینده
- Consistency

Error Handling

- ErrorBoundary در layout اصلی.
به صورت جداگانه در صورت نیاز ErrorBoundary و استفاده

- مدیریت errorهای API در react-query

استاندارد مستندسازی در docs

برای هر Feature:

```
docs/
├─ auth.md
├─ product.md
```

└─ architecture.md
└─ setup.md

محتوای هر فایل:

- هدف Feature
- APIها
- Stateها
- کاربر Flow
- Edge Caseها